Assignment 2

Unit Code: ENGR 6012 (System Vibration and Condition Assessment)

Name: S M Abul Fatha Rifat

Student Id: 21446012

School of Civil and Mechanical Engineering

Curtin University

Ans.to.the.Qs.no.1(1)

The MATLAB code is shown in Figure 1, which is planned for the visual representation of the cable force data in a cable bridge at two different allocated times. In the code, it's visible that sampling frequency is considered, also the channel number is selected based on the last digit of the student ID. It arranges two different time intervals, which are from 1000 Hrs. to 1005 Hrs., and the other from 1515 Hrs. to 1520 Hrs. The code generates two different plots to monitor the cable force conditions, as shown in Figure 2.

```
% Q1(a) - Plot cable force from two specific time intervals
% Clear everything
close all;
clear;
clc;
% Load the dataset
load('dataset_Q1.mat');
% Sampling frequency in Hz
fs = 2;
% Student ID
channel = 2;  % Last digit of 21446012
% 10:00 AM to 10:05 AM (5 mins = 300 sec × fs)
start1 = (10 * 60 * 60) * fs;
stop1 = start1 + 300 * fs;
data_segment1 = data(start1:stop1, channel);
time_segment1 = (0:length(data_segment1)-1) / fs;
% 3:15 PM to 3:20 PM
start2 = (15 * 60 * 60 + 15 * 60) * fs;
stop2 = (15 * 60 * 60 + 20 * 60) * fs;
data_segment2 = data(start2:stop2, channel);
time_segment2 = (0:length(data_segment2)-1) / fs;
% Plotting
figure;
subplot(2,1,1);
plot(time_segment1, data_segment1, 'b');
xlabel('Time (s)');
ylabel('Cable Force (kN)');
title(sprintf('Channel %d: 10:00-10:05 AM', channel));
subplot(2,1,2);
plot(time_segment2, data_segment2, 'r');
xlabel('Time (s)');
ylabel('Cable Force (kN)');
title(sprintf('Channel %d: 3:15-3:20 PM', channel));
sgtitle('Q1(1) - Cable Force Visualization');
```

Figure 1: MATLAB CODE GENERATION

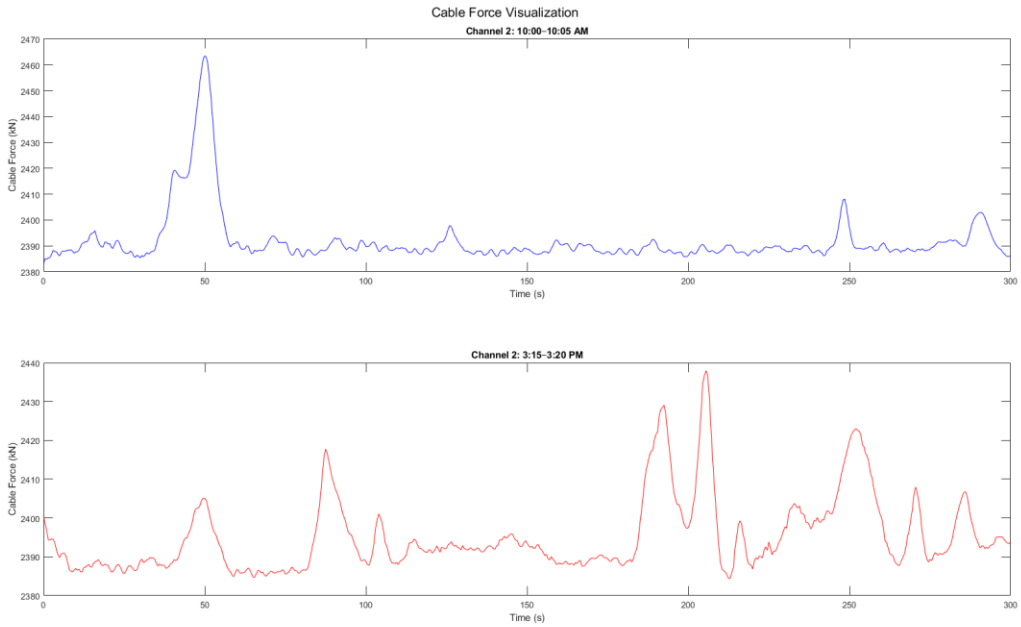

Figure 2: The Visualization of the Cable Force

Figure 2 shows two different time interval plots, as well as presenting the conditions of the cable force at 1000 Hrs. to 1005 Hrs. and 1515 Hrs. to 1520 Hrs. The first picture represents the conditions of the cable force from 1000 Hrs. to 1005 Hrs., depicting relatively stable behavior and reaching its peak around 2464 kN of cable force at approx. 50 seconds, where the second image is from 1515 Hrs. to 1520 Hrs., illustrating the frequent and large oscillations, reaching a peak around 2440 kN at around 206 seconds.

The two subplots mainly show the tension in the cable bridge at different times. The behavior of the cable is stable and smooth in the morning, but it shows dynamic behavior in the afternoon period, which is caused by different variations of the temperature, environmental effects, and traffic conditions on the structure. Mainly, these plots provide an in-depth analysis of the structure in different situations and give insightful knowledge about the conditions of the structure.

Ans.to.the.Qs.no.1(2)

The cable forces consist of three different components, such as: dead load, temperature effects, and live load. Dead load and temperature effects actively act but impact gradually over time, and dead load is considered constant because it's the weight of the structure. Live load is known as the dynamic load to the bridge, which changes much faster than the dead load and temperature effects. In this case, a high-pass filter should be used to eliminate the effects of the dead load and temperature on the structure. A high-pass filter is better for removing the low frequencies, which include dead loads and the changes of temperature, while keeping the higher frequencies, such as live loads for the movement of the vehicle over the bridge. Since the sampling rate of 2 Hz is considered, the cut-off frequency should be around 0.002 Hz to 1 Hz, which confirms the chance of removing the lower-frequency trends without impacting the desired high-frequency signal.

Ans.to.the.Qs.no.1(3)

The MATLAB code, which is shown in Figure 3, is designed to represent three different loading impacts on the cable bridge. In the previous problem, the overall cable force data is shown, but the force originates from three different components, such as the loading effects from the temperature, structure weight, and the traffic movement. The MATLAB code in Figure 3 is designed for the analysis of the three components of behavior for a better understanding of the structural response. First, the data of channel 2 is taken from the dataset_Q1.mat. Next, the Call_bandwidthpassfilter function is used to monitor the traffic-induced forces within a small period by isolating other forces and setting to keep the target frequencies. Mainly, this filtered signal is applied to observe the dynamic response of the Cable Bridge for the movement of the vehicle. Furthermore, Call_lowpassfilter is applied to remove the high frequencies from the structure load & the temperature changes, and only keep the low frequencies from these. The three individual plots are shown in Figure 4, which depicts the analysis of the original signal, traffic effects, and slowly impactful force components.

.

```
close all;
clear all;
clc;
% Load dataset
load('dataset_Q1.mat');
fs = 2;                    % Sampling rate in Hz
channel = 2;               % Based on student ID 21446012 → channel = 2
signal_raw = data(:, channel);  % Extract channel 2 data
% Plot original monitoring data
figure
subplot(3,1,1);
plot(signal_raw);
xlabel('Time Index');
ylabel('Amplitude');
xlim([1 length(signal_raw)]);
title('Original Monitoring Data');
% Define time period
T = 1 / fs;
% Filter settings
fmin = 0.002;    % Minimum frequency for band-pass filter
fmax = 1;        % Maximum frequency for band-pass filter
% Band-pass filter to isolate traffic-induced component
signal_traffic = Call_bandwidthpassfilter(signal_raw, fs, T, fmin, fmax);
subplot(3,1,2);
plot(signal_traffic);
xlabel('Time Index');
ylabel('Amplitude');
xlim([1 length(signal_traffic)]);
title('Traffic-Induced Cable Force');
% Low-pass filter to extract dead load + temperature effects
cutoff_freq = fmin;
signal_slow = Call_lowpassfilter(signal_raw, fs, cutoff_freq);
subplot(3,1,3);
plot(signal_slow);
xlabel('Time Index');
ylabel('Amplitude');
xlim([1 length(signal_slow)]);
title('Dead Load and Temperature-Induced Force');
```

Figure 3: MATLAB SCRIPT

Figure 4 represents three different subplots, where the first subplot depicts the original data, which influences the combination of three forces, such as traffic movement, structure weight, and temperature changes. That's why the first subplot exhibits larger and unpredictable amplitudes.

A band-pass filter is applied in the middle subplot by using the Call_bandwidthpassfilter function,

which isolates the traffic loads within a target frequency range from 0.002 Hz to 1 Hz. This subplot mainly focuses on the variations of high-frequency for the movements of the vehicle on the cable bridge, and the dynamic effect is observed properly for the moving transports. The Call_lowpassfilter function is applied in the last subplot to observe the effects of the low-frequency for the structure weight and the changes in the temperature. The third subplot only highlights the signal, which impacts slowly but over time, such as dead load and temperature changes. These parameters impact the structure slowly, unlike the moving vehicles, which change the response of the system instantly.

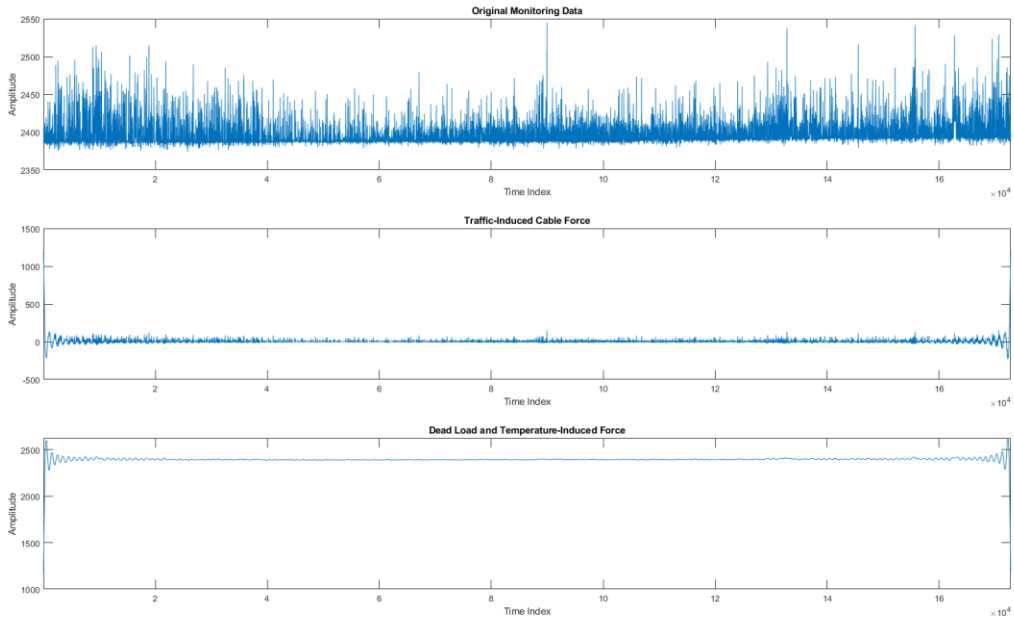

Figure 4: Observation of component forces on the Bridge

Ans.to.the.Qs.no.2

The MATLAB code is shown below:

```
% Q2 – Acceleration Signal with Time-Varying Frequency Content
close all;
clear;
clc;
% Sampling parameters
fs_rate = 400;                    % Sampling frequency [Hz]
time_step = 1 / fs_rate;
time_array = 0:time_step:45;      % 45 seconds duration
% Generate acceleration signal with time-varying frequency
acc_signal = cos(20*pi*time_array + 3*sin(0.16*pi*time_array)) + ...
             cos(13*pi*time_array + 2*sin(0.2*pi*time_array));
```

```
%% (1) Plot time-domain signal
figure;
subplot(1,3,1);
plot(time_array, acc_signal, 'k');
xlim([min(time_array) max(time_array)]);
xlabel('Time (s)');
ylabel('Acceleration (m/s²)');
title('Time Series of Time-Varying Acceleration Singal');
```

```
%% (2) Continuous Wavelet Transform (CWT)
wave_name = 'cmor8-4';            % Morlet wavelet
num_scales = 2048;                % Scale resolution
center_freq = centfrq(wave_name);
const_factor = 2 * center_freq * num_scales;
scale_index = num_scales:-1:1;
scale_vals = const_factor ./ scale_index;
% Perform CWT
wavelet_coeffs = cwt(acc_signal, scale_vals, wave_name);
freq_vector = scal2frq(scale_vals, wave_name, 1/fs_rate);
subplot(1,3,2);
imagesc(time_array, freq_vector, abs(wavelet_coeffs));
axis xy;
colorbar;
axis([min(time_array) max(time_array) 5 12]);
xlabel('Time (s)');
ylabel('Frequency (Hz)');
title('Time-Frequency via Continuous Wavelet Transform (CWT)');
```

Figure 5: MATLAB Script for Qs no 2(1) & 2(2)

```
%% (3) Synchrosqueezed Wavelet Transform (WSST)
[wsst_output, freq_wsst] = wsst(acc_signal, fs_rate);

subplot(1,3,3);
pcolor(time_array, freq_wsst, abs(wsst_output));
shading interp;
colorbar;
xlabel('Time (s)');
ylabel(' Frequency (Hz)');
title('Time-Frequency via WSST Spectrum');
axis([min(time_array) max(time_array) 5 12]);

% Adjust figure size
screen_size = get(groot, 'ScreenSize');
set(gcf, 'Position', [screen_size(1), screen_size(4)*0.25, ...
                      screen_size(3), screen_size(4)*0.5]);

sgtitle('Signal a(t) Analysis');
saveas(gcf, 'Q2_21446012_plot.png');
```

Figure 6: MATLAB Script for Qs no 2(3)

Ans.to.the.Qs.no.2(1)

Figure 5 shows the MATLAB Script, which is designed to create an acceleration signal that changes with time. Here, the sampling rate is set to 400 Hz, and the simulation is run over 45 seconds. The acc_signal visualizes the nonlinear signal by taking two cosine functions, where each cosine holds one sine function inside the logic. Here, xlim() is used to put limitations on the x-axis for adjusting the time range, and plot() arranges the whole figure to monitor the behavior of the acceleration signal over time.

Figure 7 depicts the acceleration signal, which varies with time. The signal exhibits shorter and larger nonlinear oscillations that are so frequent and that have been created by using two cosine terms. Here, the X-axis presents time, which is limited by the xlim() function, and shows the data for 45 seconds, while the Y-axis shows the acceleration signal in ms^{-2}. Figure 7 shows the intense vibration over time, and that kind of dynamic response is visible in heavy structures like cable bridges due to multiple factors. The curves of the figure represent the non-linear and frequently changing characteristics of the signal.

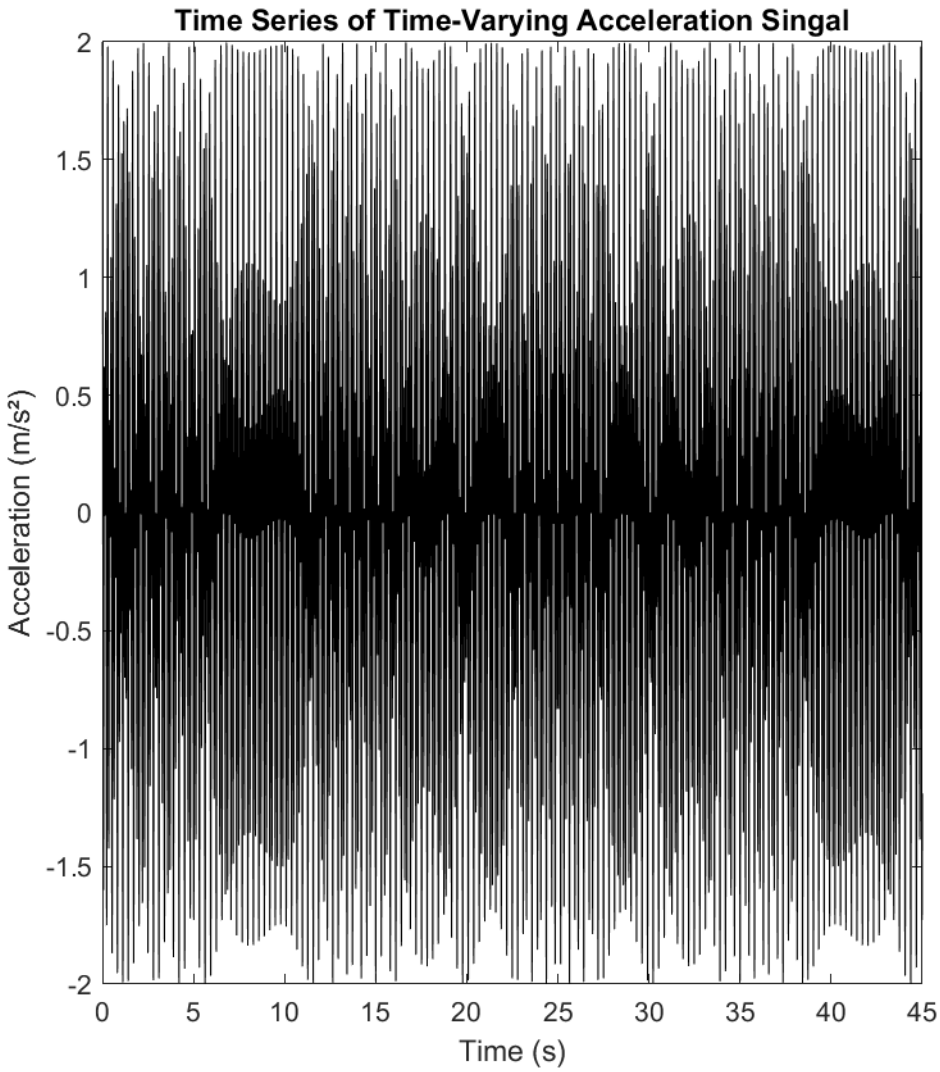

Figure 7: Time-Varying Acceleration Signal

Ans.to.the.Qs.no.2(2)

Figure 5 presents the MATLAB script, which is designed to monitor the frequency of the acceleration signal over time in the system by using a different approach, which is known as Continuous Wavelet Transform (CWT). The same signal has been created in the last task, but this time, the pattern of the output has been changed by using some different functions. Here, cmor8-4 is introduced to analyze the signal, which generates a wavelet, and larger scales (2048) have been used for in-depth analysis. Mainly, the wavelet aids in splitting the system's signal into small sections and monitors the frequency over time. Here, the centfrq() function is applied to calculate the wavelet's central frequency. Next, the cwt() function is introduced to the system to depict

how signal frequency changes with time. Furthermore, the scal2frq() function has been applied to get the numerical value of the frequency from the results of the wavelet. Lastly, the imagesc() function is used to show the response of the system.

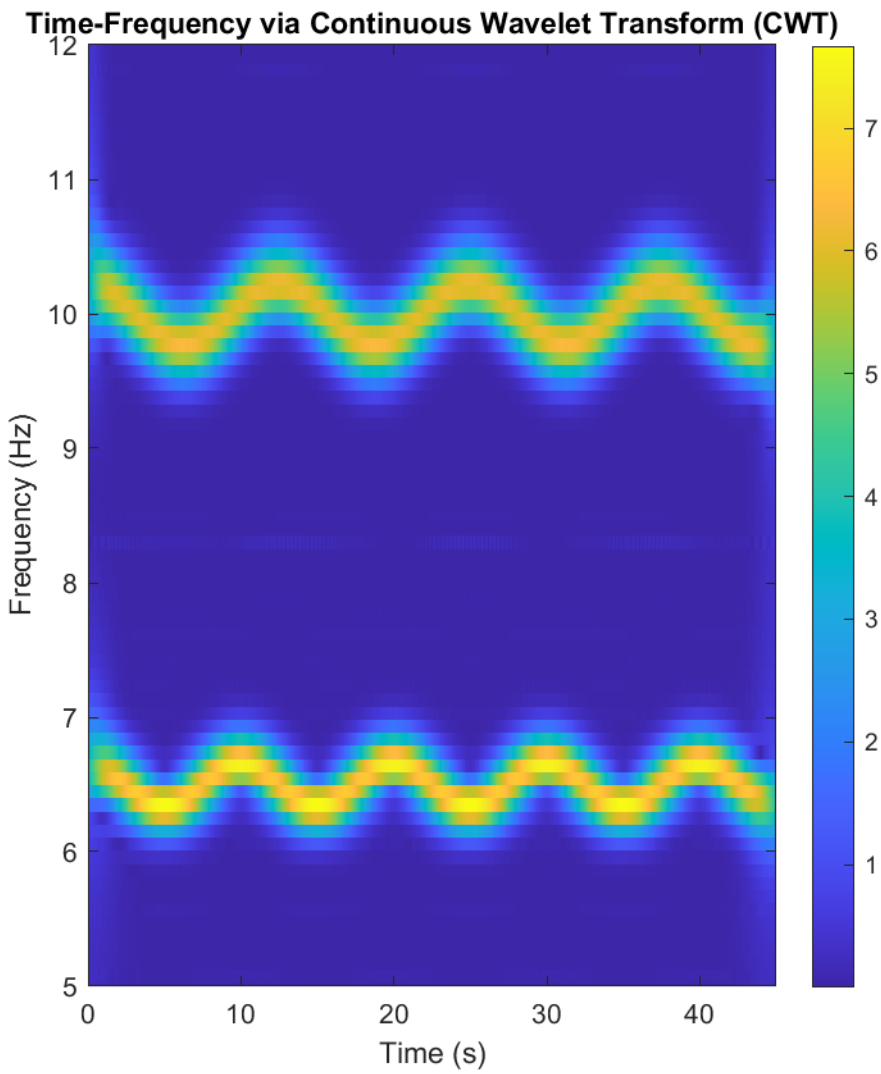

Figure 8: Time-Frequency Plot by using CWT

Figure 8 depicts the time-frequency by using the Continuous Wavelet Transform (CWT). It helps to show the response of the system and also visualize the acceleration signal with a timeframe of 45 seconds. In Figure 8, bright yellow represents the main components of the frequency, which change evenly over time. Moreover, the frequency of the signal is nonlinear, which means that the value changes over time frequently. The two different lines ensure that two separate components are applied in the system to create that curve. In short, the plot represents the behavior of the signal over time.

Ans.to.the.Qs.no.2(3)

Figure 6 represents the MATLAB Script, where the main goal is to analyze the acceleration signal over time more perfectly using the Synchrosqueezed Wavelet Transform (WSST). Here, the wsst() function has been introduced to represent the in-depth frequency analysis over time. Moreover, the wsst() function is more capable of providing better analysis than the cwt() function. Besides, pcolor() is used for color grading and shading for better frequency analysis. Furthermore, the wsst_output (time-frequency coefficients) and freq_wsst (corresponding frequency values) are the output functions, where wsst_output aids in calculating the coefficient of the time-frequency and the freq_wsst is responsible for calculating the values of frequency.

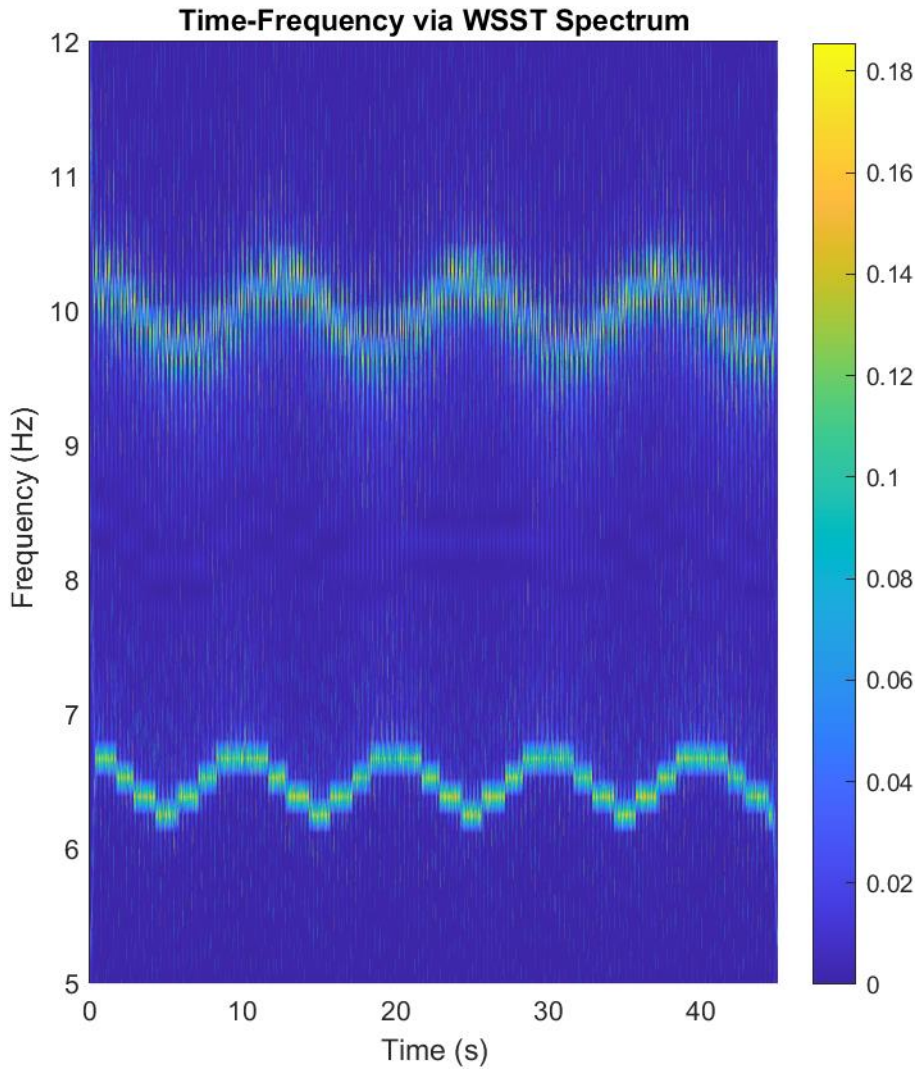

Figure 9: Time-Frequency by using WSST

Figure 9 shows the acceleration signal by using the Synchrosqueezed Wavelet Transform (WSST). This procedure provides better detailed analysis of the frequency than the Standard Wavelet Transform. In Figure 9, the X-axis shows the time in Seconds, and the Y-axis depicts the Frequency in Hz. Two Separate frequencies are visible in Figure 9, which exhibit non-stationary characteristics and address the intensity of the system's frequency every second. Besides, the brighter yellow zone indicates the stronger frequency on the system. In short, Figure 9 shows a more precise visual representation of the behavior of the frequency over time.

Ans.to.theQs.no.3(1)

```
% q3(1) - Natural Frequencies and Mode Shapes with Condition Labels

close all;
clear;
clc;

%% Parameters
samplingRate = 500;  % Sampling frequency [Hz]

for caseIndex = 0:2
    % Load case data
    caseName = sprintf('Xdd_case%d', caseIndex);
    fileName = [caseName, '.mat'];
    load(fileName);

    % Assign dataset based on case
    if caseIndex == 0
        beamData = Xdd_case0;
        caseLabel = 'Healthy Case';
        naturalFreqs = [20.4 40.2 58.6];
    elseif caseIndex == 1
        beamData = Xdd_case1;
        caseLabel = 'Damage Case 1 (30% stiffness ↓ in element 4)';
        naturalFreqs = [19 58 75.4];
    else
        beamData = Xdd_case2;
        caseLabel = 'Damage Case 2 (30% stiffness ↓ in element 16)';
        naturalFreqs = [19.4 40 56];
    end

    % FFT computation
    N = length(beamData);
    freqAxis = ([-N/2:N/2-1] * samplingRate / N);
    fftAmplitude = abs(fftshift(fft(beamData, N))) * 2 / N;

    % Raw FFT plot
    figure;
    plot(freqAxis, fftAmplitude);
    xlim([0 250]);
    xlabel('Frequency (Hz)');
    ylabel('FFT Amplitude');
    title(sprintf('Raw FFT Spectrum - %s', caseLabel));
```

Figure 10: MATLAB Script (Part 1)

```
    % Log-scaled FFT plot
    figure;
    semilogy(linspace(-samplingRate/2, samplingRate/2, N), ...
        abs(fftshift(fft(beamData, N))) * 2 / N);
    xlim([0 samplingRate / 2]);
    xlabel('Frequency (Hz)');
    ylabel('Fourier Amplitude');
    title(sprintf('Log-Scaled FFT - %s', caseLabel));

    % FDD analysis
    [Nout, fsOut, orderCount, freqGrid, UMatrix] = Call_FDD(beamData, samplingRate);

    fprintf('\n%s\n', caseLabel);
    fprintf('First three natural frequencies: %.1f, %.1f, %.1f Hz\n', naturalFreqs);

    % Pick mode shapes from selected frequencies
    for modeIndex = 1:length(naturalFreqs)
        [~, modeShapes(:, modeIndex)] = ...
            Call_FDD_Pick(Nout, fsOut, orderCount, freqGrid, UMatrix, naturalFreqs(modeIndex));
    end

    % Normalize mode shapes
    numModes = 3;
    for j = 1:numModes
        modeShapes(:, j) = modeShapes(:, j) / max(abs(modeShapes(:, j)));
    end

    % Plot mode shapes
    figure;
    hold on;
    for k = 1:numModes
        subplot(3, 1, k);
        plot(1:21, -modeShapes(:, k), '+-');
        xlabel('Node Index');
        ylabel('Normalized Amplitude');
        xlim([1 21]);
        grid on;
        title(sprintf('Mode Shape %d', k));
    end
    sgtitle(sprintf('Mode Shapes - %s', caseLabel));
end
```

Figure 11: MATLAB Script (Part 2)

Figures 10 & 11 depict the MATLAB Script, which is designed for monitoring the vibration characteristics for various structural conditions. Here, the Sampling rate is considered and loads the dataset from Xdd_case0, Xdd_case1 & Xdd_case2 to know about the natural frequency for the healthy and damaged structures. Moreover, Xdd_case1 represents the dataset for the damaged case 1, where a 30% reduction of stiffness in element 4. Furthermore, Xdd_case2 is the dataset for the damaged case 2, where 30% stiffness reduction happens in element 16, as well as Xdd_case0 represents the dataset for the healthy structures. Next, the Fast Fourier Transform (FFT) is applied to analyze the behavior of the frequency, as well as presenting the figures in logarithmic scale and normal scale. Then, Frequency Domain Decomposition (FDD) is introduced to apply the Call_FDD function to analyze the natural frequency, mode shape, and the damping ratio of the structures. Next, Call_FDD_Pick has been used to show the vibration of the structure at a specific natural frequency. The MATLAB code is generated to understand the dynamic response of the beam under different operational conditions.

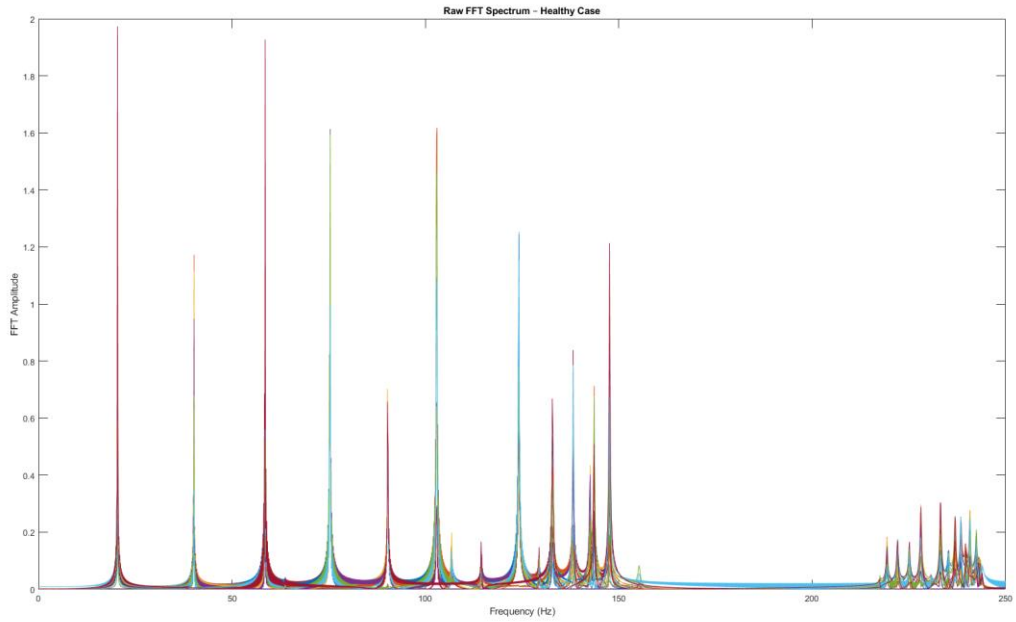

Figure 12: Raw Fast Fourier Transform Spectrum (Healthy Structure)

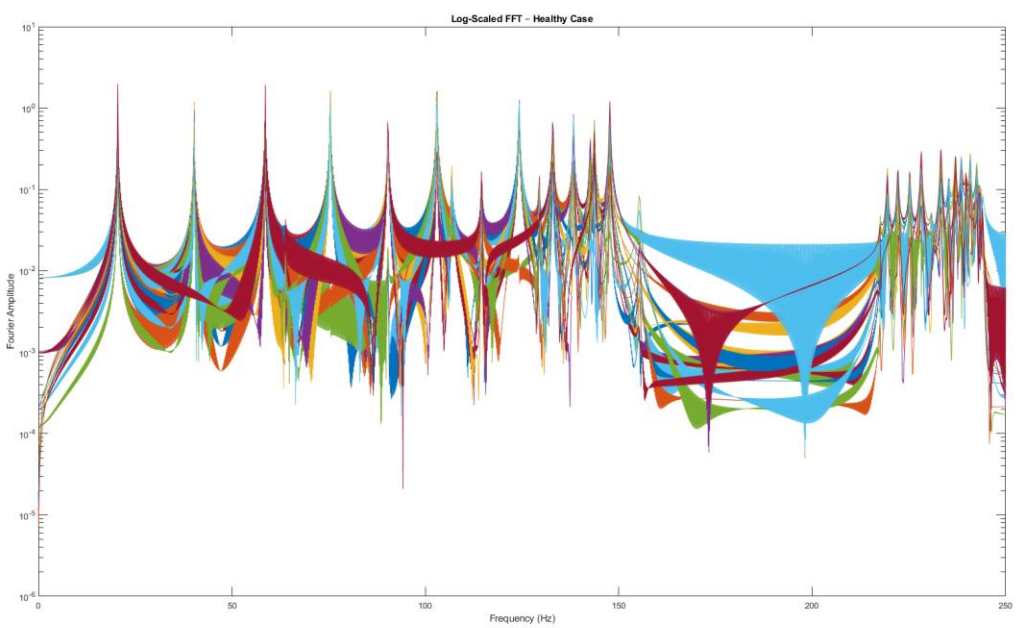

Figure 13: Log-scale Fast Fourier Transform Spectrum (Healthy Structure)

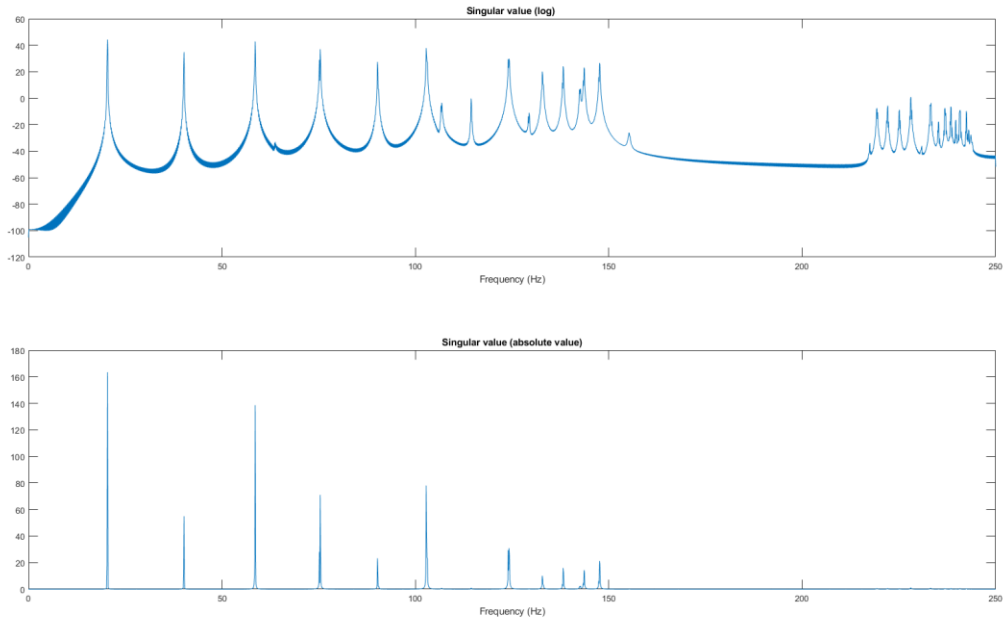

Figure 14: Singular Values for Healthy Structure

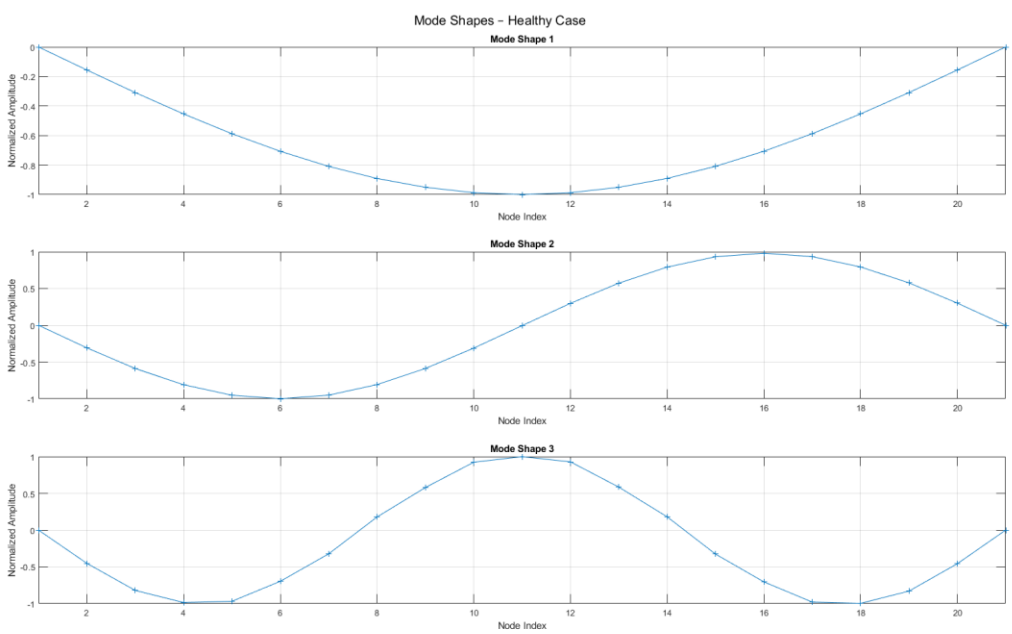

Figure 15: Mode Shapes for Healthy Structure

Figures 12 & 13 represent the raw-scale FFT & log-scale FFT to understand about the dominant frequencies by monitoring the amplitudes of the frequencies from the dynamic response of the structure. There are some peaks on the figures, which represent the natural frequencies of the beam. Here, Figure 14 illustrates the singular plot, which focuses on those natural frequencies by depicting the peaks in the plot that represent the dynamic characteristics of the modes. Figure 15 shows the mode shapes, which monitor the amplitude of the structure on the first three natural frequencies. Mainly, it represents the dynamic response of the healthy beam on its first three natural frequencies.

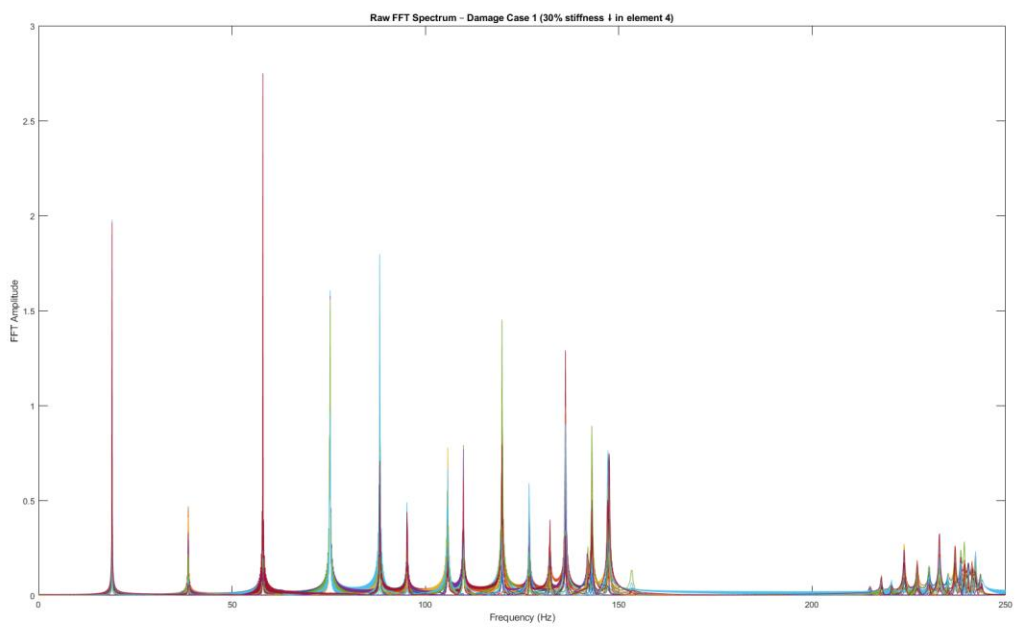

Figure 16: Raw-scale FFT Spectrum for Damaged case 1

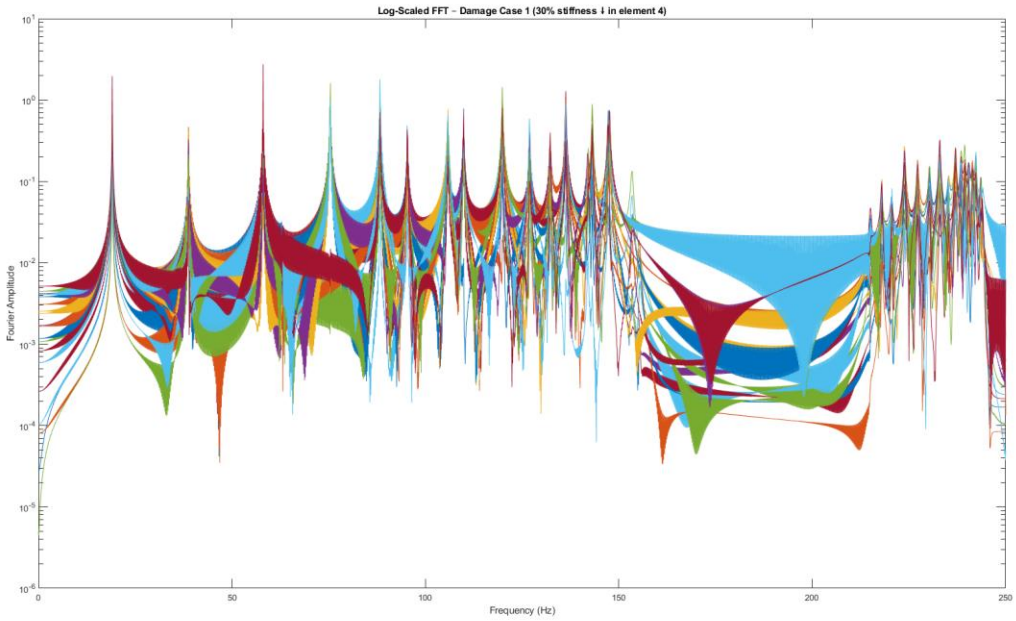

Figure 17: Log-scale FFT for Damaged Case 1

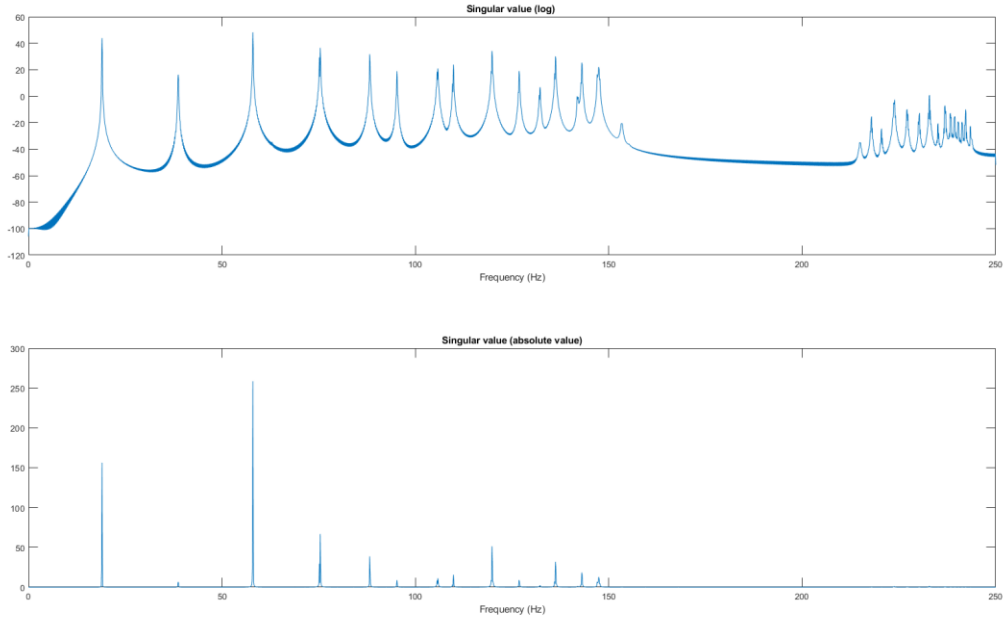

Figure 18: Singular Value for Damaged Case 1

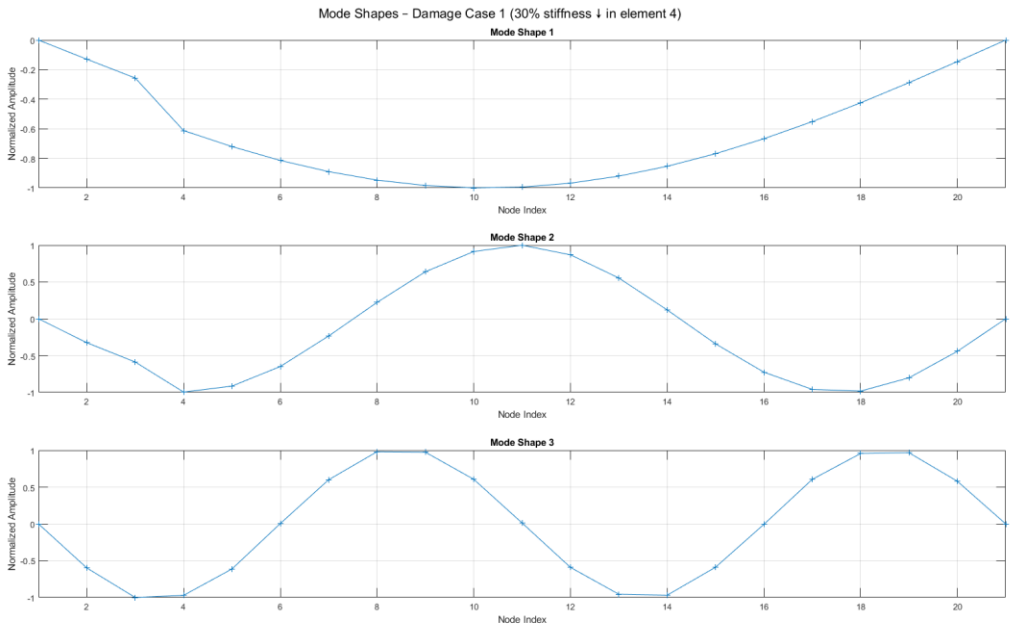

Figure 19: Mode Shapes for Damaged Case 1

Figures 16, 17, 18 & 19 represent the plots for the damaged beam, where the stiffness is 30% reduced due to the structural damage in element 4. That's why those figures exhibit different behavior compared to the healthy structures. Figure 16 represents the normal-scale Fast Fourier transform, where the amplitudes are visibly increased, as well as a bit shifting on the peak of the frequencies is noticeable, which proves that stiffness has been reduced from the structure because the vibration of the system has changed. Here, Figure 17 depicts the log-scale FFT also shows larger peak frequencies, which addresses the damage to the beam. Figure 18 represents the singular plots, where both scales, such as log and absolute, have been shown, and in both cases, peaks have been shifted, are prone to losing their sharpness in shape, which indicates that the system response has changed. Besides, the mode shape is changed and becomes more complex, as well as exhibiting that damping is also increased due to the reduction of the stiffness. Figure 19 illustrates the mode shape for the damaged beam, exhibiting a less smooth sinusoidal curve than the healthy case, and the distortion on the curve is visible at node 4, which indicates the damage occurs in element 4 on the beam.

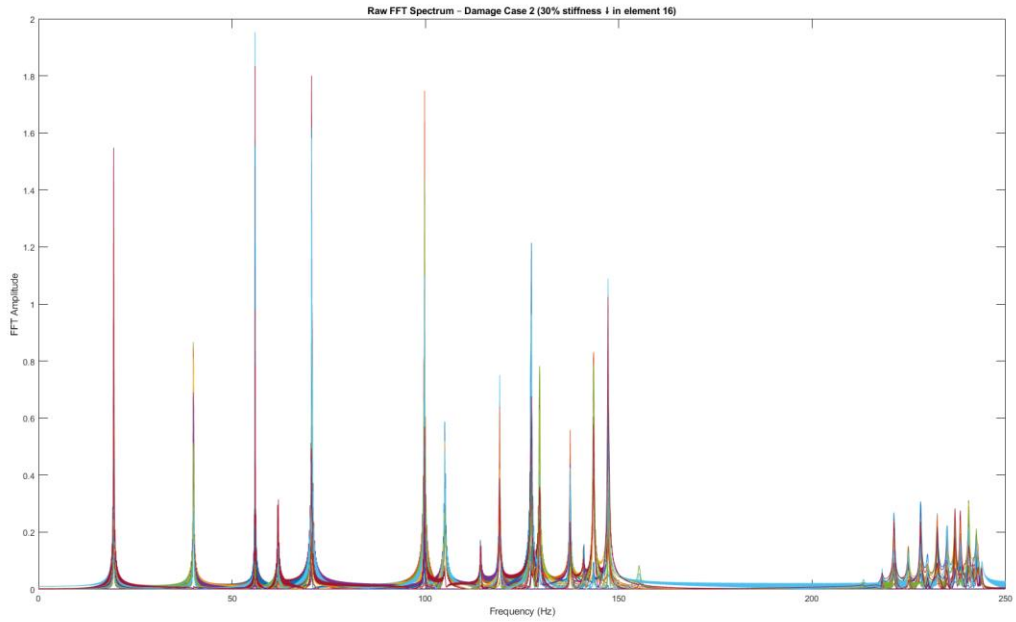

Figure 20: Normal-Scale FFT for Damaged Case 2

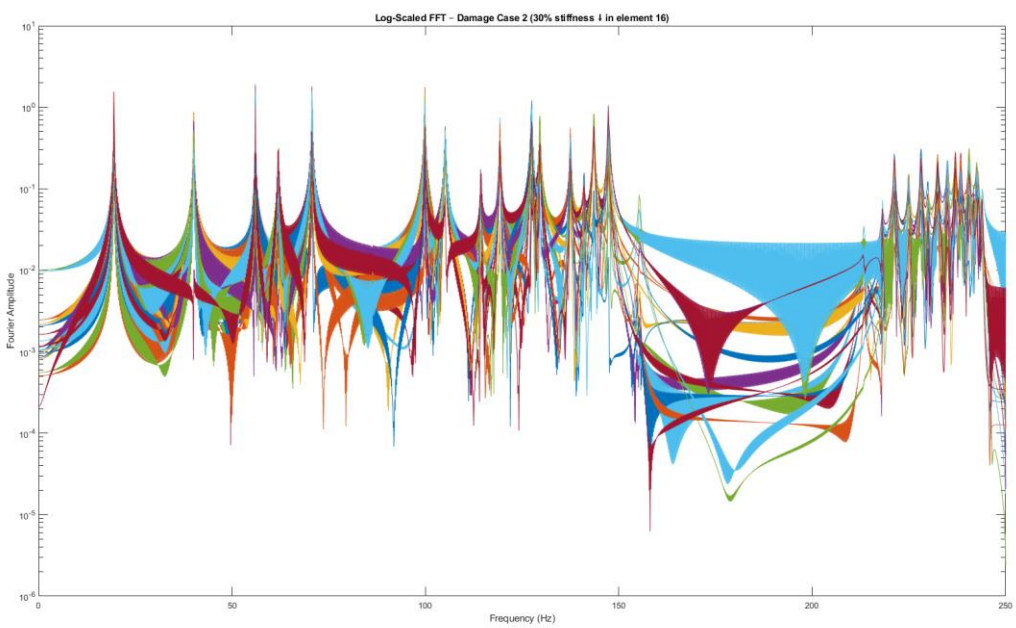

Figure 21: Log-scale FFT for Damaged Case 2

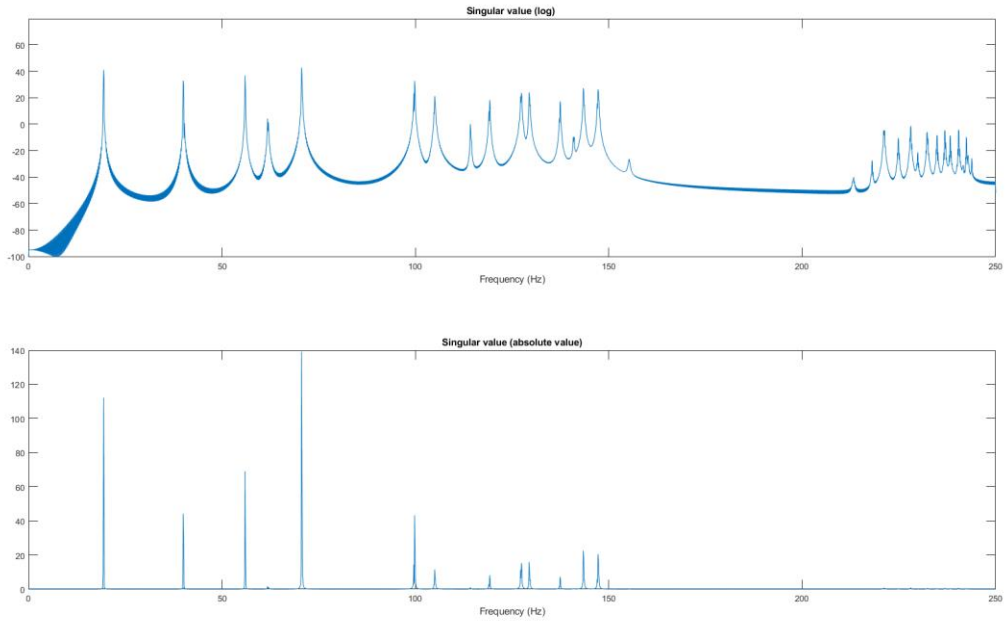

Figure 22: Singular Value for Damaged Case 2

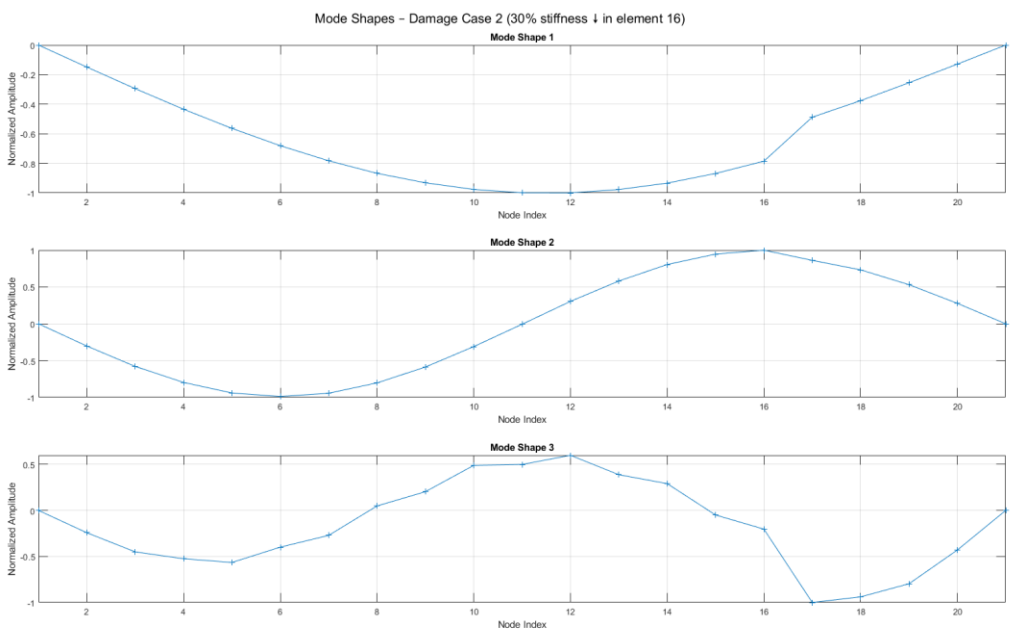

Figure 23: Mode Shape for Damaged Case 2

Figures 20, 21, 22 & 23 represent the plots for the damaged beam, where the stiffness is 30% reduced due to the structural damage in element 16. That's why these plots display different characteristics relative to the healthy beam and the damaged beam for case 1. Figure 20 illustrates the normal-scale Fast Fourier transform, where the amplitudes and peaks are visibly altered, which confirms that the behavior of the vibration has been changed due to the reduction of stiffness. Here, Figure 21 illustrates the log-scale FFT also shows multiple peak frequencies, but all of the peaks are not sharp like a healthy beam; all are prone to spread out, which addresses the damage to the beam. Figure 22 depicts the singular plots, where both scales, such as log and absolute, have been shown, and in both cases, peaks have been shifted but are clear to identify. Moreover, peaks lose their sharpness in shape as well as less regular than the healthy beam, which records different responses to the system and ensures that the system response has changed due to the damage. Besides, the mode shape is changed and becomes more complex, as well as exhibiting that damping is also increased due to the reduction of the stiffness. Figure 23 illustrates the mode shape for the damaged beam, exhibiting a less smooth sinusoidal curve because the distortion on the curve is visible at node 16, which indicates the damage occurs in element 16 on the beam, where the healthy beam exhibits a smooth sinusoidal curve.

Ans.to.the.Qs.no. 3(2)

Figures 24 & 25 depicts the MATLAB script, which applies the flexibility change technique to find out the position of the damage by monitoring the mode shapes of the healthy and damaged beam. The MATLAB code aids in calculating the flexibility matrix for cases, compares the values with others. The values that are mostly shifted address the most probable damaged beam. At first, the MATLAB code starts with loading data to get the response on the structure, and the Call_CMIF function is used to monitor the shapes of the mode and the natural frequencies. The Call_CMIF_Pick is applied to choose the shapes of the mode at specific frequencies. The max(abs(..)) is applied to normalize the mode shapes by ensuring the comparison. Next, the flexibility matrix is calculated by using the flexibility function from the chosen mode shapes and frequencies. Mainly, the MATLAB script generates the behavior of the vibration, presents the shapes of the mode, and shows the location of the damaged portion on the structure.

```
% q3(2) - Damage Detection using Flexibility Change Method
close all;
clear;
clc;
%% Parameters
fs = 500;  % Sampling frequency [Hz]
%% CASE 1: Healthy vs. Damage Case 1 (30% stiffness reduction in element 4)
% Load healthy structure data
load('Xdd_case0.mat');
dataHealthy = Xdd_case0;
% CMIF for healthy state
[nfft, n, fs, P] = Call_CMIF(dataHealthy, fs);
freqsHealthy = [20.3857, 40.2222, 58.5938];
for i = 1:length(freqsHealthy)
    [~, modeShapeHealthy(:,i)] = Call_CMIF_Pick(nfft, n, fs, P, freqsHealthy(i));
end
for i = 1:length(freqsHealthy)
    modeShapeHealthy(:,i) = modeShapeHealthy(:,i) / max(abs(modeShapeHealthy(:,i)));
end
% Load damage case 1 (element 4)
load('Xdd_case1.mat');
dataDamage1 = Xdd_case1;
[nfft, n, fs, P] = Call_CMIF(dataDamage1, fs);
freqsDamage1 = [18.9819, 38.6353, 57.9834];
for i = 1:length(freqsDamage1)
    [~, modeShapeDamage1(:,i)] = Call_CMIF_Pick(nfft, n, fs, P, freqsDamage1(i));
end
for i = 1:length(freqsDamage1)
    modeShapeDamage1(:,i) = modeShapeDamage1(:,i) / max(abs(modeShapeDamage1(:,i)));
end
% Plot mode shape comparisons (Healthy vs Damage Case 1)
figure;
subplot(2,1,1);
plot(1:21, modeShapeHealthy(:,1), '*-'); hold on;
plot(1:21, modeShapeDamage1(:,1), '+-');
xlabel('Element Index'); ylabel('Normalized Amplitude');
title('Mode 1: Healthy vs Damage Case 1 (Element 4)');
grid on; legend('Healthy', 'Damage Case 1');
subplot(2,1,2);
plot(1:21, modeShapeHealthy(:,2), '*-'); hold on;
plot(1:21, modeShapeDamage1(:,2), '+-');
xlabel('Element Index'); ylabel('Normalized Amplitude');
title('Mode 2: Healthy vs Damage Case 1 (Element 4)');
grid on; legend('Healthy', 'Damage Case 1');
```

Figure 24: MATLAB Script (Part 1)

```
% Damage localization - Case 1
figure;
truncOrder = 2:3;
flexHealthy = flexibility(modeShapeHealthy(:,truncOrder), freqsHealthy(truncOrder));
flexDamage1 = flexibility(modeShapeDamage1(:,truncOrder), freqsDamage1(truncOrder));
deltaFlex1 = flexHealthy - flexDamage1;
sigma1 = max(abs(deltaFlex1));
bar(sigma1);
xlabel('Element Index'); ylabel('Flexibility Change');
title('Damage Localization: Case 1 (Element 4)');
xticks(1:21); xlim([0 22]);
%% CASE 2: Healthy vs. Damage Case 2 (30% stiffness reduction in element 16)
load('Xdd_case2.mat');
dataDamage2 = Xdd_case2;
[nfft, n, fs, P] = Call_CMIF(dataDamage2, fs);
freqsDamage2 = [19.4702, 40.0391, 55.9692];
for i = 1:length(freqsDamage2)
    [~, modeShapeDamage2(:,i)] = Call_CMIF_Pick(nfft, n, fs, P, freqsDamage2(i));
end
for i = 1:length(freqsDamage2)
    modeShapeDamage2(:,i) = modeShapeDamage2(:,i) / max(abs(modeShapeDamage2(:,i)));
end
% Plot mode shape comparisons (Healthy vs Damage Case 2)
figure;
subplot(2,1,1);
plot(1:21, modeShapeHealthy(:,1), '*-'); hold on;
plot(1:21, modeShapeDamage2(:,1), '+-');
xlabel('Element Index'); ylabel('Normalized Amplitude');
title('Mode 1: Healthy vs Damage Case 2 (Element 16)');
grid on; legend('Healthy', 'Damage Case 2');
subplot(2,1,2);
plot(1:21, modeShapeHealthy(:,2), '*-'); hold on;
plot(1:21, modeShapeDamage2(:,2), '+-');
xlabel('Element Index'); ylabel('Normalized Amplitude');
title('Mode 2: Healthy vs Damage Case 2 (Element 16)');
grid on; legend('Healthy', 'Damage Case 2');
% Damage localization - Case 2
figure;
flexDamage2 = flexibility(modeShapeDamage2(:,truncOrder), freqsDamage2(truncOrder));
deltaFlex2 = flexHealthy - flexDamage2;
sigma2 = max(abs(deltaFlex2));
bar(sigma2);
xlabel('Element Index'); ylabel('Flexibility Change');
title('Damage Localization: Case 2 (Element 16)');
xticks(1:21); xlim([0 22]);
```

Figure 25: MATLAB Script (Part 2)

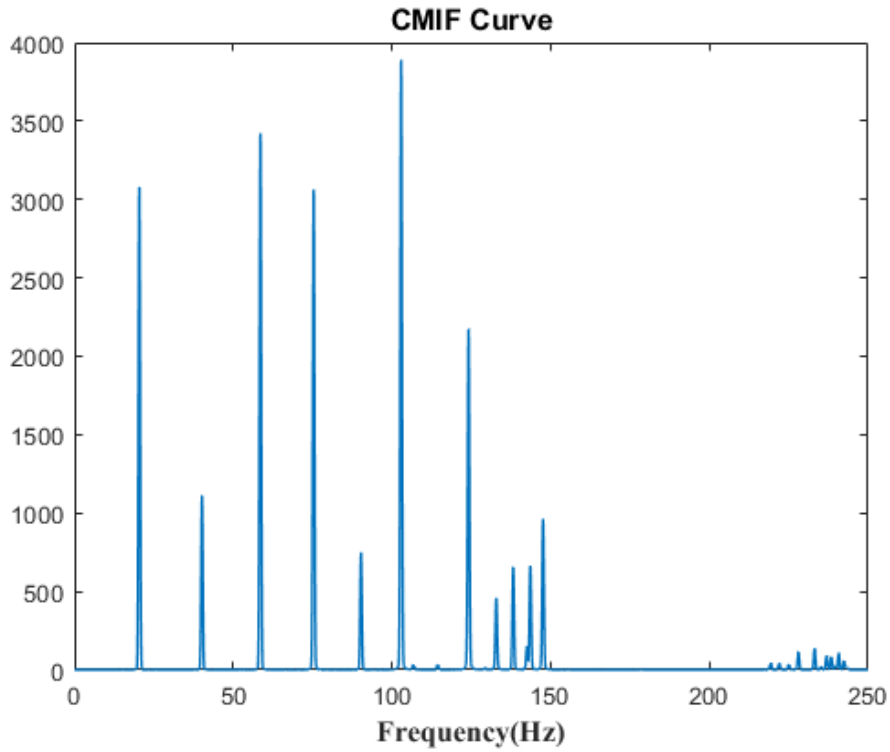

Figure 26: CMIF Curve for the Healthy Beam

Figure 26 illustrates the Complex Mode Indicator Function (CMIF) for the healthy structures. It shows Frequency on the X-axis and amplitude on the Y-axis, and tries to figure out the dominant modes of vibration. Healthy structures exhibit sharper peaks, which present the natural frequencies of the beam, which can be the ideal ones to draw the mode shapes. In Figure 26, the gaps and sharp peaks provide a clear idea that the structure is a healthy beam, and it can help to draw a reference mode shape, which will identify the location of the damaged portion on the damaged beam by comparing the mode shape.

Figure 27 illustrates the CMIF curve for the damaged case 1, where the stiffness has been reduced by 30% in element 4. It's visible in Figure 27 that the peak has been shifted significantly compared to the Healthy Beam, addressing that the dynamic behavior has changed for the damaged structure. Figure 28 represents the comparison of the mode shapes, which shows that the sinusoidal curve behaves abnormally at element index 4, as well as exhibiting slight changes in the sinusoidal curve, compared to the healthy structure. Mainly, mode shapes comparison helps to identify the damaged beam, and it can even assume the damaged location as well. Figure 29 depicts the localization bar chart, which focuses on identifying the damaged location of the beam. This plot exhibits remarkably flexibility change in the element index 4, which locates the damaged portion of the beam, and that is matched with the assumption of the comparison of mode shapes

in Figure 28. So, it's proven that the flexibility change technique is the best to identify the locations of the structural damage.

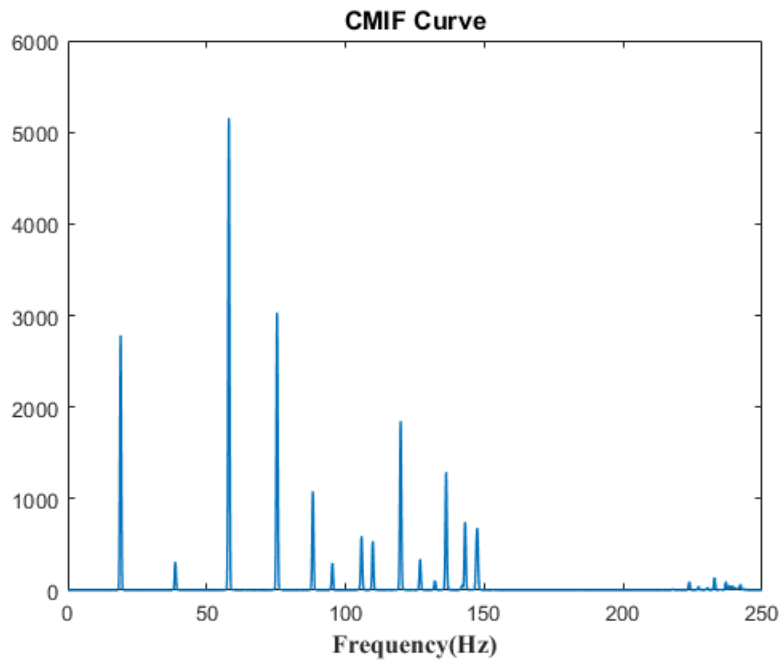

Figure 27: CMIF Curve for Damaged Beam (case 1)

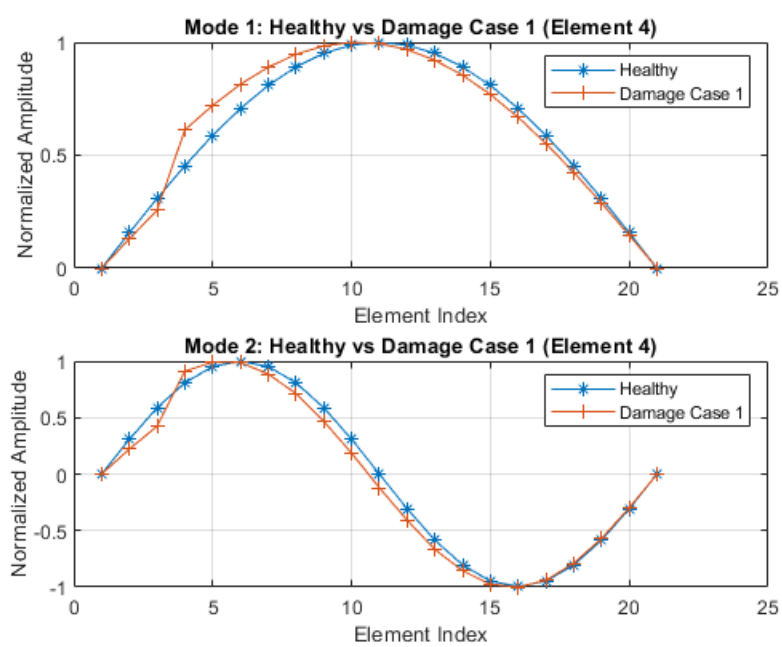

Figure 28: Mode Shapes (Healthy vs Damage Case 1)

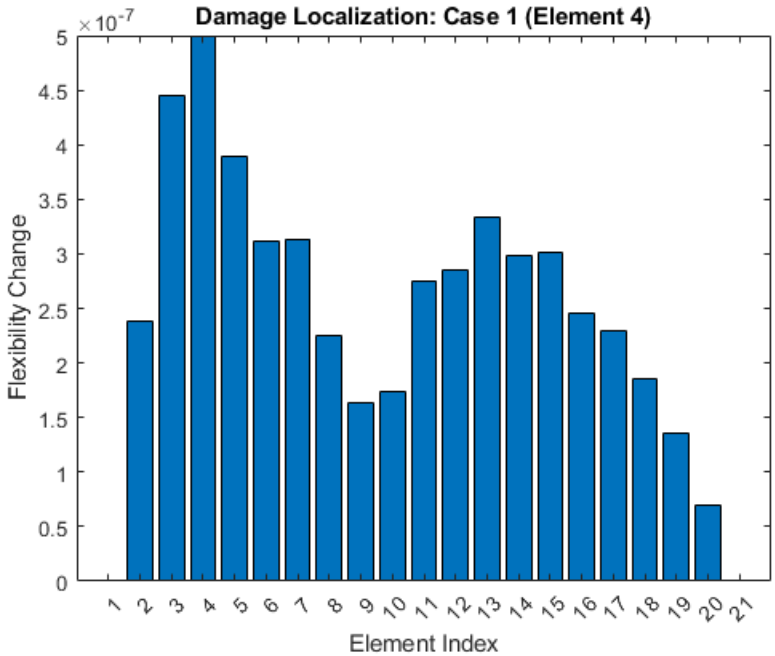

Figure 29: Damage Localization (case 1)

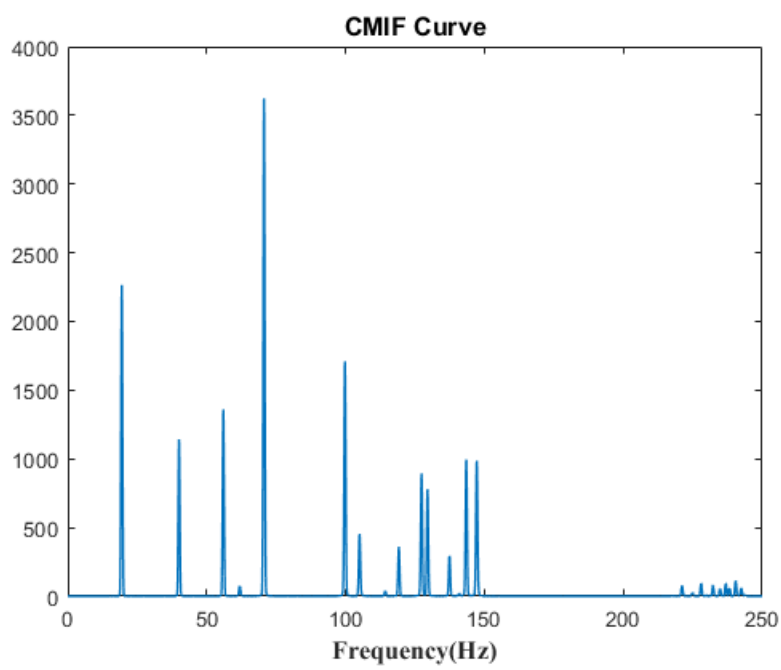

Figure 30: CMIF Curve for Damaged Beam (Case 2)

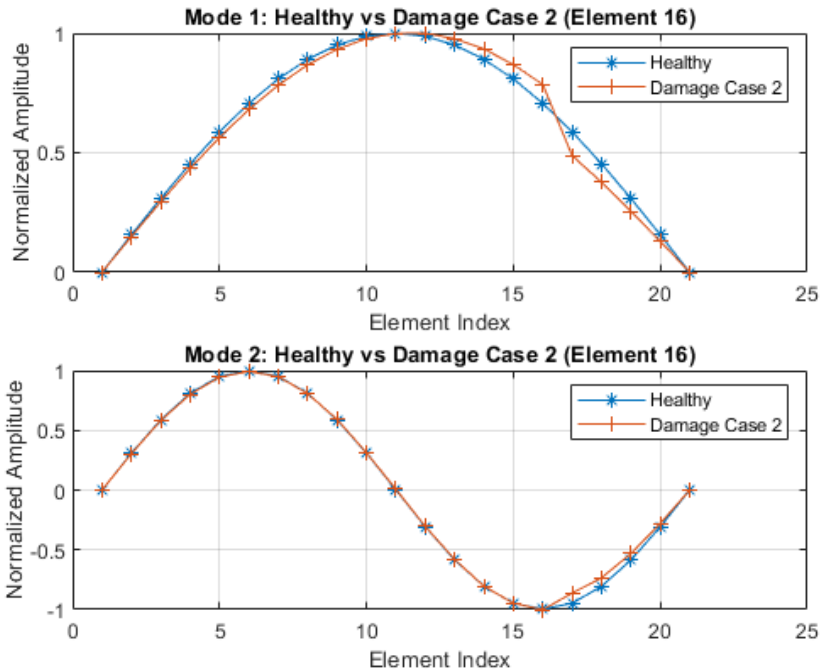

Figure 31: Mode Shapes (Healthy vs Damaged Case 2)

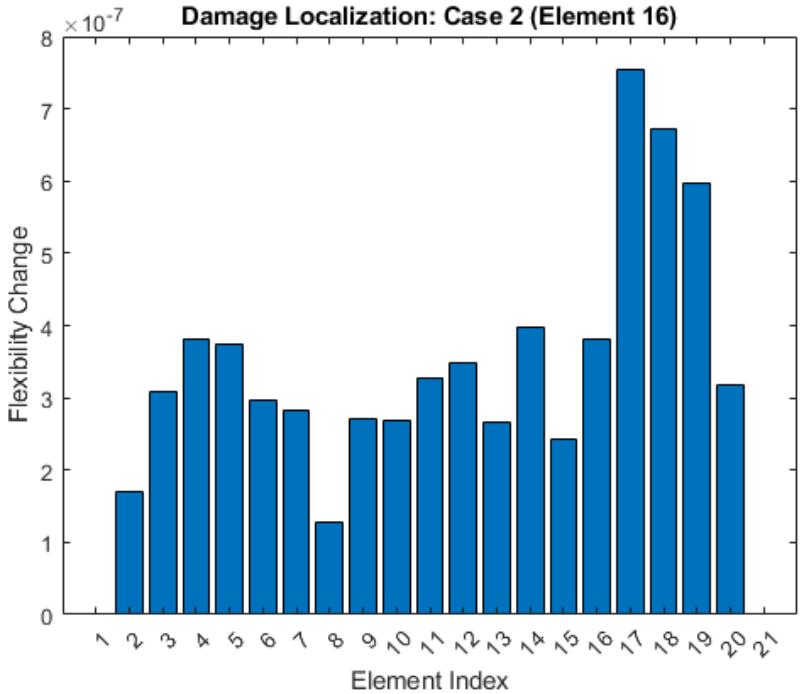

Figure 32: Damage Localization (Case 2)

Figure 30 represents the CMIF curve for the damaged case 2, where 30% reduction of stiffness in element 16. It's visible in Figure 30 that the peak has been altered, compared to the Healthy Beam, addressing that the dynamic behavior has changed for the damaged structure. Figure 31 illustrates the comparison of the mode shapes, which shows that the sinusoidal curve behaves abnormally at element index 16, as well as displaying slight changes in the sinusoidal curve, compared to the healthy structure. The changes in the sinusoidal curve confirm the reduction in stiffness due to the damaged portions. Figure 32 depicts the localization bar chart, which highlights to find out the damaged location of the beam. This plot exhibits remarkably flexibility change in the element index 16, which locates the damaged portion of the beam.

Ans.to.the.Qs.no.3(3)

```
% q3(3) - Compute global stiffness matrix (variable names only changed)
close all
clear all
clc
% Define geometric and material properties
beam_w = 0.01;
beam_h = 0.02;
inertia_val = beam_w * beam_h^3 / 12;
young_mod = 206e9;
area_val = 2e-4;
elem_len = 4 / 20;
% Local stiffness matrix for one element
k_elem = [12,        6*elem_len,     -12,        6*elem_len;
          6*elem_len, 4*elem_len^2,  -6*elem_len, 2*elem_len^2;
         -12,       -6*elem_len,      12,       -6*elem_len;
          6*elem_len, 2*elem_len^2,  -6*elem_len, 4*elem_len^2];
% Initialize global stiffness matrix (42 DOFs for 21 nodes)
k_total = zeros(42, 42);
% Assemble global stiffness matrix from element contributions
for idx = 1:20
    k_total((idx-1)*2+1:idx*2, (idx-1)*2+1:idx*2) = ...
        k_total((idx-1)*2+1:idx*2, (idx-1)*2+1:idx*2) + k_elem(1:2,1:2);
    k_total((idx-1)*2+1:idx*2, (idx*2)+1:(idx*2)+2) = ...
        k_total((idx-1)*2+1:idx*2, (idx*2)+1:(idx*2)+2) + k_elem(1:2,3:4);
    k_total((idx*2)+1:(idx*2)+2, (idx-1)*2+1:idx*2) = ...
        k_total((idx*2)+1:(idx*2)+2, (idx-1)*2+1:idx*2) + k_elem(3:4,1:2);
    k_total((idx*2)+1:(idx*2)+2, (idx*2)+1:(idx*2)+2) = ...
        k_total((idx*2)+1:(idx*2)+2, (idx*2)+1:(idx*2)+2) + k_elem(3:4,3:4);
end

% Apply material and geometry scaling (note: inertia_val was overwritten by loop if used)
k_total = (young_mod * idx / elem_len^3) * k_total;
% Scale for output and export to Excel
Global_Stiffness_Matrix = k_total / 1e13;
writematrix(Global_Stiffness_Matrix, 'kglobal.xlsx');
% === ADDED SECTION: MASS MATRIX CALCULATION ===
% Add mass matrix computation using consistent mass matrix for beam
density = 7850;  % steel [kg/m^3]
m_elem = density * area_val * elem_len / 420 * ...
         [156,     22*elem_len,    54,     -13*elem_len;
          22*elem_len,  4*elem_len^2,  13*elem_len, -3*elem_len^2;
          54,     13*elem_len,   156,    -22*elem_len;
         -13*elem_len, -3*elem_len^2, -22*elem_len,  4*elem_len^2];
% Initialize global mass matrix
mass_matrix = zeros(42, 42);
for idx = 1:20
    dof_range = (idx-1)*2+1:(idx+1)*2;
    mass_matrix(dof_range, dof_range) = mass_matrix(dof_range, dof_range) + m_elem;
end
% Export mass matrix
writematrix(mass_matrix, 'mass_matrix.xlsx');
```

Figure 33: MATLAB Script

Figure 33 shows the MATLAB Script, which is designed to calculate the global stiffness matrix and mass matrix for a beam, which is created by 21 nodes and 20 elements. First, the code defines the width and height of the beam, and the material properties, such as Young's

Modulus and density, are defined. Next, the k_elem function has been created to create a stiffness matrix for an element by using the beam theory. Then, the loop is created and the stiffness matrix is prepared to be assembled by using the K_total function, where each matrix is placed on the right side. The mass matrix has been processed in the same way to calculate its values.

The values of the Global Stiffness Matrix and Mass Matrix are given below:

Global Stiffness Matrix:
https://docs.google.com/spreadsheets/d/19FDOYxOYN4nKTIfOiNyHKpqrPMdIJuE4BHB6lVcFE5I/edit?usp=sharing

Mass Matrix:

https://docs.google.com/spreadsheets/d/1DEy-zHSOhg7wTZg47qOctRK49LZo1xwGSUqPbj8gQwY/edit?usp=sharing